Refactoring - Ejercicio 3

Mayo 2024

Cátedra

- Orientación a Objetos 2

Alumnos

- Iván Gonzalez 21816/3
- Ayrton Alvarez 21926/8

1) Mal olor: Switch statements **4**
Refactoring: Replace Conditional Logic with Polymorphism 4
2) Mal olor: Feature envy **8**
Refactoring: Extract Method 9
3) Mal olor: Speculative Generality **11**
Refactor: Inline Variable 12
4) Mal Olor: Feature Envy **13**
Refactoring: Move Method 13
5) Mal olor: Speculative Generality **15**
Refactoring: Inline variable 15
6) Mal olor: Switch Statement **16**
Refactoring: Replace Type Code with SubClasses 16
7) Mal olor: Long Method **21**
Refactoring: Extract Method 21
8) Mal olor: Inappropriate Intimacy **23**
Refactoring: Extract Method 23
9) Mal olor: Speculative Generality **25**
Refactoring: Inline Variable 25
10) Mal olor: Feature envy **26**
Refactoring: Extract Method 26
11) Mal olor: Feature envy **27**
Refactoring: Move Method 27
12) Mal olor: Long Method **29**
Refactoring: Extract Method 29
13) Mal olor: Switch Statements **31**
Refactoring: Replace Conditional with Polymorphism 31
14) Mal olor: Switch Statements **35**
Refactoring: Replace Conditional with Polymorphism 35
15) Mal olor: Message Chains **37**
Refactoring: Hide Delegate 37
16) Mal olor: Duplicate Code **38**
Refactoring: Substitute Algorithm 38
17) Mal olor: Declaración de atributo público **39**
Refactoring: Encapsulate Field 39
18) Mal olor: Declaración de atributo público **40**
Refactoring: Encapsulate Field 40
19) Mal olor: Declaración de atributo público **41**
Refactoring: Encapsulate Field 41
20) Mal olor: Speculative Generality **42**
Refactoring: Eliminar la v.i. 42
21) Mal olor: Feature Envy **44**
Refactoring: Extract Method 44
22) Mal olor: Dead Code **46**

Refactoring: Remove Parameter....46
23) Mal olor: Duplicate Code....47
Refactoring: Extract Method....47
24) Mal olor: Duplicate Code....48
Refactoring: Form Template Method....48
25) Mal olor: Nombre No Representativo....51
Refactoring: Rename Parameter....51
26) Mal olor: Nombre No Representativo....52
Refactoring: Rename Parameter....52
27) Mal olor: Nombre No Representativo....53
Refactoring: Rename Variable....53
28) Mal olor: Nombre No Representativo....54
Refactoring: Rename Variable....54
29) Mal olor: Nombre No Representativo....55
Refactoring: Rename Parameter....55
30) Mal olor: Nombre No Representativo....56
Refactoring: Rename Parameter....56

1) Mal olor: Switch statements

En la orientación a objetos, el switch representa código que puede ser repartido en otras clases y realizar el mismo comportamiento mediante el uso de polimorfismo. Para resolver este mal olor, hay que refactorizar.

Clase GestorNumeroDisponibles:

```
public String obtenerNumeroLibre() {
    String linea;
    switch (tipoGenerador) {
        case "ultimo":
            linea = lineas.last();
            lineas.remove(linea);
            return linea;
        case "primero":
            linea = lineas.first();
            lineas.remove(linea);
            return linea;
        case "random":
            linea = new ArrayList<String>(lineas)
                    .get(new Random().nextInt(lineas.size()));
            lineas.remove(linea);
            return linea;
    }
    return null;
}
```

Refactoring: Replace Conditional Logic with Polymorphism

Con este refactoring vamos a tener que crear una nueva clase abstracta TipoGenerador, que a su vez tendrá las subclases GeneradorPrimero, GeneradorUltimo y GeneradorRandom.
Después vamos a mover el método obtenerNumeroLibre() a la clase TipoGenerador como un método abstracto, para finalmente implementarlo en las subclases mencionadas.
En la clase GestorNumeroDisponibles tengo que modificar la v.i. tipoGenerador para que en lugar de ser un String sea una instancia de TipoGenerador; y el método obtenerNumeroLibre() pasará a enviar el mensaje obtenerNumeroLibre() a la variable tipoGenerador.
Así quedarían las clases:

Clase GestorNumeroDisponibles:

```java
public class GestorNumerosDisponibles {
    private SortedSet<String> lineas = new TreeSet<String>();
    private TipoGenerador tipoGenerador;

    public GestorNumerosDisponibles() {
        this.tipoGenerador = new GeneradorUltimo();
    }

    public SortedSet<String> getLineas() {
        return lineas;
    }

    public String obtenerNumeroLibre() {
        return this.tipoGenerador.obtenerNumeroLibre(lineas);
    }

    public void cambiarTipoGenerador(TipoGenerador valor) {
        this.tipoGenerador = valor;
    }
}
```

Clase TipoGenerador:

```java
public abstract class TipoGenerador {

    public abstract String obtenerNumeroLibre(SortedSet<String> lineas);

}
```

Clase GeneradorPrimero:

```
5  public class GeneradorPrimero extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         String linea = lineas.first();
11         lineas.remove(linea);
12         return linea;
13     }
14
15 }
```

Clase GeneradorUltimo:

```
4
5  public class GeneradorUltimo extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         String linea = lineas.last();
11         lineas.remove(linea);
12         return linea;
13     }
14
15 }
```

Clase GeneradorRandom:

```
7  public class GeneradorRandom extends TipoGenerador {
8
9      @Override
10     public String obtenerNumeroLibre(SortedSet<String> lineas) {
11         // TODO Auto-generated method stub
12         String linea = new ArrayList<String>(lineas)
13                 .get(new Random().nextInt(lineas.size()));
14         lineas.remove(linea);
15         return linea;
16     }
17
18 }
```

También se tuvo que modificar el test obtenerNumeroLibre():

Antes:

```
54      @Test
55      void obtenerNumeroLibre() {
56          // por defecto es el ultimo
57          assertEquals("2214444559", this.sistema.obtenerNumeroLibre());
58
59          this.sistema.getGestorNumeros().cambiarTipoGenerador("primero");
60          assertEquals("2214444554", this.sistema.obtenerNumeroLibre());
61
62          this.sistema.getGestorNumeros().cambiarTipoGenerador("random");
63          assertNotNull(this.sistema.obtenerNumeroLibre());
64      }
```

Después:

```
54      @Test
55      void obtenerNumeroLibre() {
56          // por defecto es el ultimo
57          assertEquals("2214444559", this.sistema.obtenerNumeroLibre());
58
59          this.sistema.getGestorNumeros().cambiarTipoGenerador(new GeneradorPrimero());
60          assertEquals("2214444554", this.sistema.obtenerNumeroLibre());
61
62          this.sistema.getGestorNumeros().cambiarTipoGenerador(new GeneradorRandom());
63          assertNotNull(this.sistema.obtenerNumeroLibre());
64      }
```

2) Mal olor: Feature envy

En las clases GeneradorPrimero, GeneradorUltimo y GeneradorRandom se produce envidia de atributos ya que se encargan de eliminar un dato de una colección que no les corresponde, y esta eliminación debería hacerla la clase GestorNumerosDisponibles.

Clase GeneradorUltimo:

```
4
5  public class GeneradorUltimo extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         String linea = lineas.last();
11         lineas.remove(linea);
12         return linea;
13     }
14
15 }
```

Clase GeneradorPrimero:

```
5  public class GeneradorPrimero extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         String linea = lineas.first();
11         lineas.remove(linea);
12         return linea;
13     }
14
15 }
```

Clase GeneradorRandom:

```
7  public class GeneradorRandom extends TipoGenerador {
8
9      @Override
10     public String obtenerNumeroLibre(SortedSet<String> lineas) {
11         // TODO Auto-generated method stub
12         String linea = new ArrayList<String>(lineas)
13                 .get(new Random().nextInt(lineas.size()));
14         lineas.remove(linea);
15         return linea;
16     }
17
18 }
```

Refactoring: Extract Method

Para esto extraemos del metodo obtenerNumeroLibre() de las subclases GeneradorPrimero, GeneradorUltimo y GeneradorRandom la sentencia “lineas.remove(linea)”, ya que “lineas” es una coleccion perteneciente a la clase GestorNumerosDIsponibles, esta sentencia la trasladamos al metodo obtenerNumeroLibre() de la clase GestorNumerosLibres

Clase GestorNumerosDisponibles:

```
18      public String obtenerNumeroLibre() {
19          String linea = this.tipoGenerador.obtenerNumeroLibre(lineas);
20          lineas.remove(linea);
21          return linea;
22      }
```

Clase GeneradorPrimero:

```
 5  public class GeneradorPrimero extends TipoGenerador {
 6
 7      @Override
 8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
 9          // TODO Auto-generated method stub
10          String linea = lineas.first();
11          return linea;
12      }
13
14  }
```

Clase GeneradorUltimo:

```
4
5  public class GeneradorUltimo extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         String linea = lineas.last();
11         return linea;
12     }
13
14 }
```

Clase GeneradorRandom:

```
6
7  public class GeneradorRandom extends TipoGenerador {
8
9      @Override
10     public String obtenerNumeroLibre(SortedSet<String> lineas) {
11         // TODO Auto-generated method stub
12         String linea = new ArrayList<String>(lineas)
13                 .get(new Random().nextInt(lineas.size()));
14         return linea;
15     }
16
17 }
```

3) Mal olor: Speculative Generality

En las subclases de TIpoGenerador (GeneradorPrimero, GeneradorUltimo y GeneradorRandom), se detecta una variable que solo se guarda para posibles usos futuros cuando en realidad solo se retorna en la linea siguiente, por eso podemos refactorizar.

Clase GeneradorPrimero:

```
5  public class GeneradorPrimero extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         String linea = lineas.first();
11         return linea;
12     }
13
14 }
```

Clase GeneradorUltimo:

```
4
5  public class GeneradorUltimo extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         String linea = lineas.last();
11         return linea;
12     }
13
14 }
```

Clase GeneradorRandom:

```
6
7  public class GeneradorRandom extends TipoGenerador {
8
9      @Override
10     public String obtenerNumeroLibre(SortedSet<String> lineas) {
11         // TODO Auto-generated method stub
12         String linea = new ArrayList<String>(lineas)
13                 .get(new Random().nextInt(lineas.size()));
14         return linea;
15     }
16
17 }
```

Refactor: Inline Variable

Lo que realizamos en este refactor es dejar de asignar las busquedas de numeros disponibles (primero, ultimo y random) a una variable “linea” para poder directamente retornar los resultados de cada busqueda requerida.

Clase GeneradorPrimero:

```
5  public class GeneradorPrimero extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         return lineas.first();
11     }
12
13 }
```

Clase GeneradorUltimo:

```
4
5  public class GeneradorUltimo extends TipoGenerador {
6
7      @Override
8      public String obtenerNumeroLibre(SortedSet<String> lineas) {
9          // TODO Auto-generated method stub
10         return lineas.last();
11     }
12
13 }
```

Clase GeneradorRandom:

```
7  public class GeneradorRandom extends TipoGenerador {
8
9      @Override
10     public String obtenerNumeroLibre(SortedSet<String> lineas) {
11         // TODO Auto-generated method stub
12         return new ArrayList<String>(lineas)
13                 .get(new Random().nextInt(lineas.size()));
14     }
15
16 }
```

4) Mal Olor: Feature Envy

En la clase Empresa, podemos visualizar en el metodo agregarNumeroTelefono() como se solicita a la clase GestorNumerosDisponibles su coleccion de lineas para consultar si contiene o no una linea especifica recibida como parametro, en caso de no existir se agrega la linea recibida como parametro a la coleccion propia de la clase GestorNumerosDisponibles lo cual genera acoplamiento entre las clases.

Clase Empresa:

```
13
14      public boolean agregarNumeroTelefono(String str) {
15          boolean encontre = guia.getLineas().contains(str);
16          if (!encontre) {
17              guia.getLineas().add(str);
18              encontre= true;
19              return encontre;
20          }
21          else {
22              encontre= false;
23              return encontre;
24          }
25      }
```

Refactoring: Move Method

Para solucionar este mal olor hay que mover el método a la clase GestorNumeroDisponibles, y el método que queda en Empresa simplemente se encargara de pasar el mensaje a su variable “guia”.

Clase Empresa:

```
13
14      public boolean agregarNumeroTelefono(String str) {
15          return guia.agregarNumeroTelefono(str);
16      }
```

Clase GestorNumerosDisponibles:

```
27
28      public boolean agregarNumeroTelefono(String str) {
29          boolean encontre = getLineas().contains(str);
30          if (!encontre) {
31              getLineas().add(str);
32              encontre= true;
33              return encontre;
34          }
35          else {
36              encontre= false;
37              return encontre;
38          }
39      }
40  }
```

5) Mal olor: Speculative Generality

En la clase GestorNumerosDisponibles se detecta la variable “encontre”, que solo se lo usa para asignarle un valor y utilizarlo como condición, y despues para asignarle otro valor y retornarlo, por lo que no es necesaria en el método.

Clase GestorNumerosDisponibles:

```
27
28      public boolean agregarNumeroTelefono(String str) {
29          boolean encontre = getLineas().contains(str);
30          if (!encontre) {
31              getLineas().add(str);
32              encontre= true;
33              return encontre;
34          }
35          else {
36              encontre= false;
37              return encontre;
38          }
39      }
40  }
```

Refactoring: Inline variable

Con esto vamos a dejar de utilizar la variable “encontre” para utilizar directamente la expresión “getLineas().contains(str)” como condición del “if”, y para retornar directamente los valores “true” o “false” en el “return”

Clase GestorNumerosDisponibles:

```
27
28      public boolean agregarNumeroTelefono(String str) {
29          if (!getLineas().contains(str)) {
30              getLineas().add(str);
31              return true;
32          }
33          else {
34              return false;
35          }
36      }
```

6) Mal olor: Switch Statement

En la clase Empresa tenemos el metodo registrarUsuario() que utiliza una estructura de control if para verificar de que “tipo” es el cliente (fisica ó juridica).

Clase Empresa:

```
21
22      public Cliente registrarUsuario(String data, String nombre, String tipo) {
23          Cliente var = new Cliente();
24          if (tipo.equals("fisica")) {
25              var.setNombre(nombre);
26              String tel = this.obtenerNumeroLibre();
27              var.setTipo(tipo);
28              var.setNumeroTelefono(tel);
29              var.setDNI(data);
30          }
31          else if (tipo.equals("juridica")) {
32              String tel = this.obtenerNumeroLibre();
33              var.setNombre(nombre);
34              var.setTipo(tipo);
35              var.setNumeroTelefono(tel);
36              var.setCuit(data);
37          }
38          clientes.add(var);
39          return var;
40      }
```

Refactoring: Replace Type Code with SubClasses

Con este refactoring crearíamos 2 nuevas clases ClienteFisico y ClienteJuridico, quedando la clase Cliente como abstracta, luego procedemos a mover el comportamiento relevante de la clase Cliente a las subclases mencionadas.

Clase Empresa:

```java

    public Cliente registrarUsuarioFisico(String data, String nombre) {
        ClienteFisico var = new ClienteFisico();
        var.setNombre(nombre);
        String tel = this.obtenerNumeroLibre();
        var.setNumeroTelefono(tel);
        var.setDNI(data);
        clientes.add(var);
        return var;
    }

    public Cliente registrarUsuarioJuridico(String data, String nombre) {
        ClienteJuridico var = new ClienteJuridico();
        String tel = this.obtenerNumeroLibre();
        var.setNombre(nombre);
        var.setNumeroTelefono(tel);
        var.setCuit(data);
        clientes.add(var);
        return var;
    }
```

Clase Cliente:

```java
public abstract class Cliente {
    public List<Llamada> llamadas = new ArrayList<Llamada>();
    private String nombre;
    private String numeroTelefono;

    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getNumeroTelefono() {
        return numeroTelefono;
    }
    public void setNumeroTelefono(String numeroTelefono) {
        this.numeroTelefono = numeroTelefono;
    }

}
```

Clase ClienteFisico:

```java
public class ClienteFisico extends Cliente {

    private String dni;

    public String getDNI() {
        return dni;
    }
    public void setDNI(String dni) {
        this.dni = dni;
    }
}
```

Clase ClienteJuridico:

```java
public class ClienteJuridico extends Cliente {

    private String cuit;

    public String getCuit() {
        return cuit;
    }
    public void setCuit(String cuit) {
        this.cuit = cuit;
    }
}
```

Se tuvo que modificar también el siguiente método de la clase Empresa para que, en lugar de preguntar por el atributo “tipo”, pregunta por la clase del objeto ***(esto es para que funcionen los tests)***

Antes Clase Empresa:

```
44
45    public double calcularMontoTotalLlamadas(Cliente cliente) {
46        double c = 0;
47        for (Llamada l : cliente.llamadas) {
48            double auxc = 0;
49            if (l.getTipoDeLlamada() == "nacional") {
50                // el precio es de 3 pesos por segundo más IVA sin adicional por establecer la llamada
51                auxc += l.getDuracion() * 3 + (l.getDuracion() * 3 * 0.21);
52            } else if (l.getTipoDeLlamada() == "internacional") {
53                // el precio es de 150 pesos por segundo más IVA más 50 pesos por establecer la llamada
54                auxc += l.getDuracion() * 150 + (l.getDuracion() * 150 * 0.21) + 50;
55            }
56
57            if (cliente.getTipo() == "fisica") {
58                auxc -= auxc*descuentoFis;
59            } else if(cliente.getTipo() == "juridica") {
60                auxc -= auxc*descuentoJur;
61            }
62            c += auxc;
63        }
64        return c;
65    }
```

Despues Clase Empresa:

```
44
45    public double calcularMontoTotalLlamadas(Cliente cliente) {
46        double c = 0;
47        for (Llamada l : cliente.llamadas) {
48            double auxc = 0;
49            if (l.getTipoDeLlamada() == "nacional") {
50                // el precio es de 3 pesos por segundo más IVA sin adicional por establecer la llamada
51                auxc += l.getDuracion() * 3 + (l.getDuracion() * 3 * 0.21);
52            } else if (l.getTipoDeLlamada() == "internacional") {
53                // el precio es de 150 pesos por segundo más IVA más 50 pesos por establecer la llamada
54                auxc += l.getDuracion() * 150 + (l.getDuracion() * 150 * 0.21) + 50;
55            }
56
57            if (cliente instanceof ClienteFisico) {
58                auxc -= auxc*descuentoFis;
59            } else if(cliente instanceof ClienteJuridico) {
60                auxc -= auxc*descuentoJur;
61            }
62            c += auxc;
63        }
64        return c;
65    }
```

Tambien se tuvo que modificar los tests debido al cambio de nombre de métodos y al cambio de parámetros.

Antes:

```
21
22    @Test
23    void testcalcularMontoTotalLlamadas() {
24        Cliente emisorPersonaFisca = sistema.registrarUsuario("11555666", "Brendan Eich" , "fisica");
25        Cliente remitentePersonaFisica = sistema.registrarUsuario("00000001", "Doug Lea" , "fisica");
26        Cliente emisorPersonaJuridica = sistema.registrarUsuario("17555222", "Nvidia Corp" , "juridica");
27        Cliente remitentePersonaJuridica = sistema.registrarUsuario("25765432", "Sun Microsystems" , "juridica");
```

```
43
44    @Test
45    void testAgregarUsuario() {
46        assertEquals(this.sistema.cantidadDeUsuarios(), 0);
47        this.sistema.agregarNumeroTelefono("2214444558");
48        Cliente nuevaPersona = this.sistema.registrarUsuario("2444555","Alan Turing", "fisica");
49
```

Despues:

```
21
22      @Test
23      void testcalcularMontoTotalLlamadas() {
24          Cliente emisorPersonaFisca = sistema.registrarUsuarioFisico("11555666", "Brendan Eich");
25          Cliente remitentePersonaFisica = sistema.registrarUsuarioFisico("00000001", "Doug Lea");
26          Cliente emisorPersonaJuridica = sistema.registrarUsuarioJuridico("17555222", "Nvidia Corp");
27          Cliente remitentePersonaJuridica = sistema.registrarUsuarioJuridico("25765432", "Sun Microsystems");
```

```
43
44      @Test
45      void testAgregarUsuario() {
46          assertEquals(this.sistema.cantidadDeUsuarios(), 0);
47          this.sistema.agregarNumeroTelefono("2214444558");
48          Cliente nuevaPersona = this.sistema.registrarUsuarioFisico("2444555","Alan Turing");
49
```

7) Mal olor: Long Method

En la clase Empresa vemos como los métodos (registrarUsuarioFisico y registrarUsuarioJuridico), realizan mas de una tarea no respetando la alta cohesion, en este caso crean una instancia de cliente y ambos metodos lo agregan a la lista de clientes.

Clase Empresa:

```
21
22      public Cliente registrarUsuarioFisico(String data, String nombre) {
23          ClienteFisico var = new ClienteFisico();
24          var.setNombre(nombre);
25          String tel = this.obtenerNumeroLibre();
26          var.setNumeroTelefono(tel);
27          var.setDNI(data);
28          clientes.add(var);
29          return var;
30      }
31
32      public Cliente registrarUsuarioJuridico(String data, String nombre) {
33          ClienteJuridico var = new ClienteJuridico();
34          String tel = this.obtenerNumeroLibre();
35          var.setNombre(nombre);
36          var.setNumeroTelefono(tel);
37          var.setCuit(data);
38          clientes.add(var);
39          return var;
40      }
```

Refactoring: Extract Method

Para refactorizar el codigo anterior podemos extraer la sentencia donde agregamos el cliente a la lista de clientes en un metodo nuevo (agregarCliente) el cual se llamaría dentro de los metodos registrarUsuarioFisico y registrarUsuarioJuridico

Clase Empresa:

```
21
22      public Cliente registrarUsuarioFisico(String data, String nombre) {
23          ClienteFisico var = new ClienteFisico();
24          var.setNombre(nombre);
25          String tel = this.obtenerNumeroLibre();
26          var.setNumeroTelefono(tel);
27          var.setDNI(data);
28          agregarUsuario(var);
29          return var;
30      }
31
32      public Cliente registrarUsuarioJuridico(String data, String nombre) {
33          ClienteJuridico var = new ClienteJuridico();
34          String tel = this.obtenerNumeroLibre();
35          var.setNombre(nombre);
36          var.setNumeroTelefono(tel);
37          var.setCuit(data);
38          agregarUsuario(var);
39          return var;
40      }
```

```
    private void agregarUsuario(Cliente cliente) {
        clientes.add(cliente);
    }
```

8) Mal olor: Inappropriate Intimacy

En los siguientes métodos, la clase Empresa instancia un objeto y setea sus campos uno por uno, en lugar de dejar que el objeto mismo se setee a través de su constructor.

Clase Empresa:

```
21
22      public Cliente registrarUsuarioFisico(String data, String nombre) {
23          ClienteFisico var = new ClienteFisico();
24          var.setNombre(nombre);
25          String tel = this.obtenerNumeroLibre();
26          var.setNumeroTelefono(tel);
27          var.setDNI(data);
28          agregarUsuario(var);
29          return var;
30      }
31
32      public Cliente registrarUsuarioJuridico(String data, String nombre) {
33          ClienteJuridico var = new ClienteJuridico();
34          String tel = this.obtenerNumeroLibre();
35          var.setNombre(nombre);
36          var.setNumeroTelefono(tel);
37          var.setCuit(data);
38          agregarUsuario(var);
39          return var;
40      }
```

Refactoring: Extract Method

La solución es extraer todos los set que realiza la clase Empresa, y moverlos en el constructor de la clase Cliente junto con sus subclases, y utilizar el nuevo constructor en el método original.

Clase Empresa:

```
21
22      public Cliente registrarUsuarioFisico(String data, String nombre) {
23          String tel = this.obtenerNumeroLibre();
24          ClienteFisico var = new ClienteFisico(nombre, tel, data);
25          agregarUsuario(var);
26          return var;
27      }
28
29      public Cliente registrarUsuarioJuridico(String data, String nombre) {
30          String tel = this.obtenerNumeroLibre();
31          ClienteJuridico var = new ClienteJuridico(nombre, tel, data);
32          agregarUsuario(var);
33          return var;
34      }
```

Clase Cliente:

```java
public abstract class Cliente {
    public List<Llamada> llamadas = new ArrayList<Llamada>();
    private String nombre;
    private String numeroTelefono;

    public Cliente(String nombre, String numeroTelefono) {
        this.nombre = nombre;
        this.numeroTelefono = numeroTelefono;
    }
```

Clase ClienteFisico:

```java
public class ClienteFisico extends Cliente {

    private String dni;

    public ClienteFisico(String nombre, String numeroTelefono, String dni) {
        super(nombre, numeroTelefono);
        this.dni = dni;
    }
```

Clase ClienteJuridico:

```java
public class ClienteJuridico extends Cliente {

    private String cuit;

    public ClienteJuridico(String nombre, String numeroTelefono, String cuit) {
        super(nombre, numeroTelefono);
        this.cuit = cuit;
    }
```

9) Mal olor: Speculative Generality

En los siguientes métodos de la clase Empresa se utiliza la variable “tel”, que solo se utiliza para enviarla al constructor.

Clase Empresa:

```
21
22      public Cliente registrarUsuarioFisico(String data, String nombre) {
23          String tel = this.obtenerNumeroLibre();
24          ClienteFisico var = new ClienteFisico(nombre, tel, data);
25          agregarUsuario(var);
26          return var;
27      }
28
29      public Cliente registrarUsuarioJuridico(String data, String nombre) {
30          String tel = this.obtenerNumeroLibre();
31          ClienteJuridico var = new ClienteJuridico(nombre, tel, data);
32          agregarUsuario(var);
33          return var;
34      }
```

Refactoring: Inline Variable

La solución es eliminar la variable “tel”, y dejar la sentencia “this.obtenerNumeroLibre()” en los parámetros del constructor.

Clase Empresa:

```
22      public Cliente registrarUsuarioFisico(String data, String nombre) {
23          ClienteFisico var = new ClienteFisico(nombre, this.obtenerNumeroLibre(), data);
24          agregarUsuario(var);
25          return var;
26      }
27
28      public Cliente registrarUsuarioJuridico(String data, String nombre) {
29          ClienteJuridico var = new ClienteJuridico(nombre, this.obtenerNumeroLibre(), data);
30          agregarUsuario(var);
31          return var;
32      }
```

10) Mal olor: Feature envy

En el siguiente método de la clase Empresa se produce envidia de atributos en la sentencia “origen.llamadas.add(llamada)”, ya que Empresa toma la lista de llamadas de un Cliente y agrega una Llamada, en lugar de dejarle esa tarea a la clase Cliente.

Clase Empresa:

```
37
38      public Llamada registrarLlamada(Cliente origen, Cliente destino, String t, int duracion) {
39          Llamada llamada = new Llamada(t, origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
40          llamadas.add(llamada);
41          origen.llamadas.add(llamada);
42          return llamada;
43      }
```

Refactoring: Extract Method

Para solucionarlo hay que extraer la sentencia y llevarla a la Clase Cliente para que sea un método de la misma. A su vez la clase Empresa enviará un mensaje al Cliente utilizando el nuevo método declarado. Quedaría de esta forma:

Clase Empresa:

```
38      public Llamada registrarLlamada(Cliente origen, Cliente destino, String t, int duracion) {
39          Llamada llamada = new Llamada(t, origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
40          llamadas.add(llamada);
41          origen.registrarLlamada(llamada);
42          return llamada;
43      }
```

Clase Cliente:

```
28      public void registrarLlamada(Llamada llamada) {
29          llamadas.add(llamada);
30      }
```

11) Mal olor: Feature envy

En el siguiente método de la clase Empresa se produce envidia de atributos, ya que calcula el monto total de llamadas de un cliente tomando su lista de llamadas y obteniendo el monto de cada llamada. Esta debería ser una tarea de la clase Cliente, y no de la clase Empresa.

Clase Empresa:

```
44
45      public double calcularMontoTotalLlamadas(Cliente cliente) {
46          double c = 0;
47          for (Llamada l : cliente.llamadas) {
48              double auxc = 0;
49              if (l.getTipoDeLlamada() == "nacional") {
50                  // el precio es de 3 pesos por segundo más IVA sin adicional por establecer la llamada
51                  auxc += l.getDuracion() * 3 + (l.getDuracion() * 3 * 0.21);
52              } else if (l.getTipoDeLlamada() == "internacional") {
53                  // el precio es de 150 pesos por segundo más IVA más 50 pesos por establecer la llamada
54                  auxc += l.getDuracion() * 150 + (l.getDuracion() * 150 * 0.21) + 50;
55              }
56
57              if (cliente instanceof ClienteFisico) {
58                  auxc -= auxc*descuentoFis;
59              } else if(cliente instanceof ClienteJuridico) {
60                  auxc -= auxc*descuentoJur;
61              }
62              c += auxc;
63          }
64          return c;
65      }
```

Refactoring: Move Method

La solución es mover el método a la clase Cliente, y dejar en el método de la clase Empresa solamente el envío de mensaje al cliente.

Clase Empresa:

```
42
43      public double calcularMontoTotalLlamadas(Cliente cliente) {
44          return cliente.calcularMontoTotalLlamadas();
45      }
```

Clase Cliente:

```
34
35    public double calcularMontoTotalLlamadas() {
36        double c = 0;
37        for (Llamada l : llamadas) {
38            double auxc = 0;
39            if (l.getTipoDeLlamada() == "nacional") {
40                // el precio es de 3 pesos por segundo más IVA sin adicional por establecer la llamada
41                auxc += l.getDuracion() * 3 + (l.getDuracion() * 3 * 0.21);
42            } else if (l.getTipoDeLlamada() == "internacional") {
43                // el precio es de 150 pesos por segundo más IVA más 50 pesos por establecer la llamada
44                auxc += l.getDuracion() * 150 + (l.getDuracion() * 150 * 0.21) + 50;
45            }
46
47            if (this instanceof ClienteFisico) {
48                auxc -= auxc*descuentoFis;
49            } else if(this instanceof ClienteJuridico) {
50                auxc -= auxc*descuentoJur;
51            }
52            c += auxc;
53        }
54        return c;
55    }
```

También se tienen que mover las v.i. “descuentoJur” y “descuentoFis” desde la clase Empresa hasta la clase Cliente para que siga funcionando el método.

Antes Clase Empresa:

```
 6 public class Empresa {
 7     private List<Cliente> clientes = new ArrayList<Cliente>();
 8     private List<Llamada> llamadas = new ArrayList<Llamada>();
 9     private GestorNumerosDisponibles guia = new GestorNumerosDisponibles();
10
11     static double descuentoJur = 0.15;
12     static double descuentoFis = 0;
13
```

Despues Clase Empresa:

```
 6 public class Empresa {
 7     private List<Cliente> clientes = new ArrayList<Cliente>();
 8     private List<Llamada> llamadas = new ArrayList<Llamada>();
 9     private GestorNumerosDisponibles guia = new GestorNumerosDisponibles();
10
11
12     public boolean agregarNumeroTelefono(String str) {
13         return guia.agregarNumeroTelefono(str);
14     }
```

Clase Cliente:

```
 6 public abstract class Cliente {
 7     public List<Llamada> llamadas = new ArrayList<Llamada>();
 8     private String nombre;
 9     private String numeroTelefono;
10
11     static double descuentoJur = 0.15;
12     static double descuentoFis = 0;
```

12) Mal olor: Long Method

En el siguiente método de la clase Cliente realiza el cálculo del precio de una llamada, luego aplica descuento y lo va acumulando en una variable, y realiza esto para cada llamada. Esto provoca que el método sea muy largo y realice demasiadas tareas.

Clase Cliente:

```
34
35      public double calcularMontoTotalLlamadas() {
36          double c = 0;
37          for (Llamada l : llamadas) {
38              double auxc = 0;
39              if (l.getTipoDeLlamada() == "nacional") {
40                  // el precio es de 3 pesos por segundo más IVA sin adicional por establecer la llamada
41                  auxc += l.getDuracion() * 3 + (l.getDuracion() * 3 * 0.21);
42              } else if (l.getTipoDeLlamada() == "internacional") {
43                  // el precio es de 150 pesos por segundo más IVA más 50 pesos por establecer la llamada
44                  auxc += l.getDuracion() * 150 + (l.getDuracion() * 150 * 0.21) + 50;
45              }
46
47              if (this instanceof ClienteFisico) {
48                  auxc -= auxc*descuentoFis;
49              } else if(this instanceof ClienteJuridico) {
50                  auxc -= auxc*descuentoJur;
51              }
52              c += auxc;
53          }
54          return c;
55      }
```

Refactoring: Extract Method

La solución es extraer el calculo del precio y la aplicación del descuento en dos métodos aparte, para así reducir el tamaño del método original. A su vez se tuvo que, en el caso de “calcularPrecioLlamada()” enviar la llamada como parámetro y declarar una variable temporal “auxc”, y en “aplicarDescuento()” se tuvo que pasar como parámetro la variable “auxc” del método original.

Clase Cliente:

```
34
35      public double calcularMontoTotalLlamadas() {
36          double c = 0;
37          for (Llamada l : llamadas) {
38              double auxc = 0;
39              auxc = calcularPrecioLlamada(l);
40              auxc = aplicarDescuento(auxc);
41              c += auxc;
42          }
43          return c;
44      }
45
46      private double calcularPrecioLlamada(Llamada l) {
47          double auxc = 0;
48          if (l.getTipoDeLlamada() == "nacional") {
49              // el precio es de 3 pesos por segundo más IVA sin adicional por establecer la llamada
50              auxc += l.getDuracion() * 3 + (l.getDuracion() * 3 * 0.21);
51          } else if (l.getTipoDeLlamada() == "internacional") {
52              // el precio es de 150 pesos por segundo más IVA más 50 pesos por establecer la llamada
53              auxc += l.getDuracion() * 150 + (l.getDuracion() * 150 * 0.21) + 50;
54          }
55          return auxc;
56      }
57
58      private double aplicarDescuento(double auxc) {
59          if (this instanceof ClienteFisico) {
60              auxc -= auxc*descuentoFis;
61          } else if(this instanceof ClienteJuridico) {
62              auxc -= auxc*descuentoJur;
63          }
64          return auxc;
65      }
66
```

13) Mal olor: Switch Statements

En el siguiente método de la clase Cliente se presenta un “if” que pregunta por el tipo de una llamada, y a partir de ahí realiza un cálculo diferente. Esto es una violación al diseño orientado a objetos, ya que cada clase debería ser capaz de realizar su tarea sin necesidad de preguntar por su tipo.

Clase Cliente:

```
45
46      private double calcularPrecioLlamada(Llamada l) {
47          double auxc = 0;
48          if (l.getTipoDeLlamada() == "nacional") {
49              // el precio es de 3 pesos por segundo más IVA sin adicional por establecer la llamada
50              auxc += l.getDuracion() * 3 + (l.getDuracion() * 3 * 0.21);
51          } else if (l.getTipoDeLlamada() == "internacional") {
52              // el precio es de 150 pesos por segundo más IVA más 50 pesos por establecer la llamada
53              auxc += l.getDuracion() * 150 + (l.getDuracion() * 150 * 0.21) + 50;
54          }
55          return auxc;
56      }
```

Refactoring: Replace Conditional with Polymorphism

La solución es declarar las subclases “LlamadaNacional” y “LlamadaInternacional” de la clase “Llamada”, y esta última pasará a ser una clase abstracta.
Después vamos a mover el método “calcularPrecioLlamada()” a la clase “Llamada” como método abstracto, y las subclases la implementarán.
También se eliminará la v.i. “tipoDeLlamada” de la clase “Llamada” y su getter.

Clase Cliente:

```
46      private double calcularPrecioLlamada(Llamada l) {
47          double auxc = 0;
48          auxc = l.calcularPrecioLlamada();
49          return auxc;
50      }
```

Clase Llamada:

```java
public abstract class Llamada {
    private String origen;
    private String destino;
    private int duracion;

    public Llamada(String origen, String destino, int duracion) {
        this.origen= origen;
        this.destino= destino;
        this.duracion = duracion;
    }

    public String getRemitente() {
        return destino;
    }

    public int getDuracion() {
        return this.duracion;
    }

    public String getOrigen() {
        return origen;
    }

    public abstract double calcularPrecioLlamada();
}
```

Clase LlamadaNacional:

```java
public class LlamadaNacional extends Llamada {


    public LlamadaNacional(String origen, String destino, int duracion) {
        super(origen, destino, duracion);
        // TODO Auto-generated constructor stub
    }

    @Override
    public double calcularPrecioLlamada() {
        // TODO Auto-generated method stub
        return getDuracion() * 3 + (getDuracion() * 3 * 0.21);
    }

}
```

Clase LlamadaInternacional:

```java
public class LlamadaInternacional extends Llamada {

    public LlamadaInternacional(String origen, String destino, int duracion) {
        super(origen, destino, duracion);
        // TODO Auto-generated constructor stub
    }

    @Override
    public double calcularPrecioLlamada() {
        // TODO Auto-generated method stub
        return getDuracion() * 150 + (getDuracion() * 150 * 0.21) + 50;
    }

}
```

Esto provoca a su vez que se tenga que modificar la clase Empresa, ya que el método "registrarLlamada" pasará a ser dos métodos llamados "registrarLlamadaNacional" y "registrarLlamadaInternacional".

Antes Clase Empresa:

```java
    public Llamada registrarLlamada(Cliente origen, Cliente destino, String t, int duracion) {
        Llamada llamada = new Llamada(t, origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
        llamadas.add(llamada);
        origen.registrarLlamada(llamada);
        return llamada;
    }
```

Despues Clase Empresa:

```java
    public Llamada registrarLlamadaNacional(Cliente origen, Cliente destino, int duracion) {
        Llamada llamada = new LlamadaNacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
        llamadas.add(llamada);
        origen.registrarLlamada(llamada);
        return llamada;
    }

    public Llamada registrarLlamadaInternacional(Cliente origen, Cliente destino, int duracion) {
        Llamada llamada = new LlamadaInternacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
        llamadas.add(llamada);
        origen.registrarLlamada(llamada);
        return llamada;
    }
```

Esto, a su vez, provoca que se tenga que modificar los tests debido al cambio de nombre de los métodos y al cambio de parámetros.

Antes:

```
21
22    @Test
23    void testcalcularMontoTotalLlamadas() {
24        Cliente emisorPersonaFisca = sistema.registrarUsuarioFisico("11555666", "Brendan Eich");
25        Cliente remitentePersonaFisica = sistema.registrarUsuarioFisico("00000001", "Doug Lea");
26        Cliente emisorPersonaJuridica = sistema.registrarUsuarioJuridico("17555222", "Nvidia Corp");
27        Cliente remitentePersonaJuridica = sistema.registrarUsuarioJuridico("25765432", "Sun Microsystems");
28
29        this.sistema.registrarLlamada(emisorPersonaJuridica, remitentePersonaFisica, "nacional", 10);
30        this.sistema.registrarLlamada(emisorPersonaJuridica, remitentePersonaFisica, "internacional", 8);
31        this.sistema.registrarLlamada(emisorPersonaJuridica, remitentePersonaJuridica, "nacional", 5);
32        this.sistema.registrarLlamada(emisorPersonaJuridica, remitentePersonaJuridica, "internacional", 7);
33        this.sistema.registrarLlamada(emisorPersonaFisca, remitentePersonaFisica, "nacional", 15);
34        this.sistema.registrarLlamada(emisorPersonaFisca, remitentePersonaFisica, "internacional", 45);
35        this.sistema.registrarLlamada(emisorPersonaFisca, remitentePersonaJuridica, "nacional", 13);
36        this.sistema.registrarLlamada(emisorPersonaFisca, remitentePersonaJuridica, "internacional", 17);
37
38        assertEquals(11454.64, this.sistema.calcularMontoTotalLlamadas(emisorPersonaFisca), 0.01);
39        assertEquals(2445.40, this.sistema.calcularMontoTotalLlamadas(emisorPersonaJuridica), 0.01);
40        assertEquals(0, this.sistema.calcularMontoTotalLlamadas(remitentePersonaFisica));
41        assertEquals(0, this.sistema.calcularMontoTotalLlamadas(remitentePersonaJuridica));
42    }
43
```

Despues:

```
21
22    @Test
23    void testcalcularMontoTotalLlamadas() {
24        Cliente emisorPersonaFisca = sistema.registrarUsuarioFisico("11555666", "Brendan Eich");
25        Cliente remitentePersonaFisica = sistema.registrarUsuarioFisico("00000001", "Doug Lea");
26        Cliente emisorPersonaJuridica = sistema.registrarUsuarioJuridico("17555222", "Nvidia Corp");
27        Cliente remitentePersonaJuridica = sistema.registrarUsuarioJuridico("25765432", "Sun Microsystems");
28
29        this.sistema.registrarLlamadaNacional(emisorPersonaJuridica, remitentePersonaFisica, 10);
30        this.sistema.registrarLlamadaInternacional(emisorPersonaJuridica, remitentePersonaFisica, 8);
31        this.sistema.registrarLlamadaNacional(emisorPersonaJuridica, remitentePersonaJuridica, 5);
32        this.sistema.registrarLlamadaInternacional(emisorPersonaJuridica, remitentePersonaJuridica, 7);
33        this.sistema.registrarLlamadaNacional(emisorPersonaFisca, remitentePersonaFisica, 15);
34        this.sistema.registrarLlamadaInternacional(emisorPersonaFisca, remitentePersonaFisica, 45);
35        this.sistema.registrarLlamadaNacional(emisorPersonaFisca, remitentePersonaJuridica, 13);
36        this.sistema.registrarLlamadaInternacional(emisorPersonaFisca, remitentePersonaJuridica, 17);
37
38        assertEquals(11454.64, this.sistema.calcularMontoTotalLlamadas(emisorPersonaFisca), 0.01);
39        assertEquals(2445.40, this.sistema.calcularMontoTotalLlamadas(emisorPersonaJuridica), 0.01);
40        assertEquals(0, this.sistema.calcularMontoTotalLlamadas(remitentePersonaFisica));
41        assertEquals(0, this.sistema.calcularMontoTotalLlamadas(remitentePersonaJuridica));
42    }
```

14) Mal olor: Switch Statements

En el siguiente método de la clase Cliente se utiliza un “if” preguntando por la clase de un objeto, y dependiendo del mismo realiza un descuento u otro.

Clase Cliente:

```
51
52      private double aplicarDescuento(double auxc) {
53          if (this instanceof ClienteFisico) {
54              auxc -= auxc*descuentoFis;
55          } else if(this instanceof ClienteJuridico) {
56              auxc -= auxc*descuentoJur;
57          }
58          return auxc;
59      }
```

Refactoring: Replace Conditional with Polymorphism

La solución es desarmar el condicional y dejar que las clases mismas apliquen el descuento que les corresponde. Para esto es necesario que el método original se haga abstracto y protected, y que las subclases implementen el descuento correspondiente. También hay que mover las v.i. “descuentoFis” y “descuentoJur” a las clases correspondientes.

Clase Cliente:

```
48
49      protected abstract double aplicarDescuento(double auxc);
50
```

Clase ClienteFisico:

```
3  public class ClienteFisico extends Cliente {
4
5      private String dni;
6
7      static double descuentoFis = 0;
8
9      public ClienteFisico(String nombre, String numeroTelefono, String dni) {
13
14     public String getDNI() {
17
18     public void setDNI(String dni) {
21
22     @Override
23     protected double aplicarDescuento(double auxc) {
24         // TODO Auto-generated method stub
25         return auxc - auxc*descuentoFis;
26     }
27 }
```

Clase ClienteJuridico:

```
3  public class ClienteJuridico extends Cliente {
4
5      private String cuit;
6
7      static double descuentoJur = 0.15;
8
9      public ClienteJuridico(String nombre, String numeroTelefono, String cuit) {
13
14     public String getCuit() {
17
18     public void setCuit(String cuit) {
21
22     @Override
23     protected double aplicarDescuento(double auxc) {
24         // TODO Auto-generated method stub
25         return auxc - auxc*descuentoJur;
26     }
27 }
```

15) Mal olor: Message Chains

En la clase Cliente el método calcularMontoTotalLlamadas() llama al método calcularPrecioLlamada() que lo único que hace es enviar un mensaje al parámetro que le envían y retornar el resultado, cuando directamente esta tarea la podría realizar el primer método.

Clase Cliente:

```
31
32      public double calcularMontoTotalLlamadas() {
33          double c = 0;
34          for (Llamada l : llamadas) {
35              double auxc = 0;
36              auxc = calcularPrecioLlamada(l);
37              auxc = aplicarDescuento(auxc);
38              c += auxc;
39          }
40          return c;
41      }
42
43      private double calcularPrecioLlamada(Llamada l) {
44          double auxc = 0;
45          auxc = l.calcularPrecioLlamada();
46          return auxc;
47      }
```

Refactoring: Hide Delegate

La solución es eliminar el método calcularPrecioLlamada() y dejar que el método calcularMontoTotalLlamadas() haga esa tarea.

Clase Cliente:

```
31
32      public double calcularMontoTotalLlamadas() {
33          double c = 0;
34          for (Llamada l : llamadas) {
35              double auxc = 0;
36              auxc = l.calcularPrecioLlamada();
37              auxc = aplicarDescuento(auxc);
38              c += auxc;
39          }
40          return c;
41      }
```

16) Mal olor: Duplicate Code

En el siguiente método de la clase Cliente se recorre una colección aplicando una serie de métodos para luego retornar un resultado, lo mismo que realiza un “stream()”.

Clase Cliente:

```
31
32      public double calcularMontoTotalLlamadas() {
33          double c = 0;
34          for (Llamada l : llamadas) {
35              double auxc = 0;
36              auxc = l.calcularPrecioLlamada();
37              auxc = aplicarDescuento(auxc);
38              c += auxc;
39          }
40          return c;
41      }
```

Refactoring: Substitute Algorithm

La solución es reemplazar este algoritmo con un stream(), ya que realiza un mejor trabajo. El mismo traerá el precio de cada llamada, le aplicará el descuento a cada precio y finalmente sumará todo y lo retornará.

Clase Cliente:

```
31
32      public double calcularMontoTotalLlamadas() {
33          return llamadas.stream()
34                  .mapToDouble(l -> l.calcularPrecioLlamada())
35                  .map(precio -> aplicarDescuento(precio))
36                  .sum();
37      }
```

17) Mal olor: Declaración de atributo público

En la clase Cliente, el atributo “llamadas” es público. Esto mismo viola el encapsulamiento de la clase.

Clase Cliente:

```
5
6  public abstract class Cliente {
7      public List<Llamada> llamadas = new ArrayList<Llamada>();
8      private String nombre;
9      private String numeroTelefono;
```

Refactoring: Encapsulate Field

La solución es volverla privada.

Clase Cliente:

```
5
6  public abstract class Cliente {
7      private List<Llamada> llamadas = new ArrayList<Llamada>();
8      private String nombre;
9      private String numeroTelefono;
```

18) Mal olor: Declaración de atributo público

En la clase ClienteFisico, el atributo “descuentoFis” es público. Esto mismo viola el encapsulamiento de la clase.

Clase ClienteFisico:

```
3  public class ClienteFisico extends Cliente {
4
5      private String dni;
6
7      static double descuentoFis = 0;
```

Refactoring: Encapsulate Field

La solución es volverla privada.

Clase ClienteFisico:

```
3  public class ClienteFisico extends Cliente {
4
5      private String dni;
6
7      private static double descuentoFis = 0;
```

19) Mal olor: Declaración de atributo público

En la clase ClienteJuridico, el atributo “descuentoJur” es público. Esto mismo viola el encapsulamiento de la clase.

Clase ClienteJuridico:

```
3  public class ClienteJuridico extends Cliente {
4
5      private String cuit;
6
7      static double descuentoJur = 0.15;
```

Refactoring: Encapsulate Field

La solución es volverla privada.

Clase ClienteJuridico:

```
3  public class ClienteJuridico extends Cliente {
4
5      private String cuit;
6
7      private static double descuentoJur = 0.15;
```

20) Mal olor: Speculative Generality

La v.i. “llamadas” de la clase Empresa no se utiliza, salvo para agregarle elementos a la lista. Además, para obtener las llamadas simplemente podemos utilizar la lista de llamadas que tiene cada Cliente, por lo que la información está repetida.

Clase Empresa:

```
5
6  public class Empresa {
7      private List<Cliente> clientes = new ArrayList<Cliente>();
8      private List<Llamada> llamadas = new ArrayList<Llamada>();
9      private GestorNumerosDisponibles guia = new GestorNumerosDisponibles();
10
```

Refactoring: Eliminar la v.i.

La solución es eliminar “llamadas”. Esto va a llevar a que se modifiquen los métodos “registrarLlamadaNacional()” y “registrarLlamadaInternacional()”, teniendo que eliminar la sentencia “llamadas.add(llamada)”

Clase Empresa:

```
5
6  public class Empresa {
7      private List<Cliente> clientes = new ArrayList<Cliente>();
8      private GestorNumerosDisponibles guia = new GestorNumerosDisponibles();
9
```

Esto va a llevar a que se modifiquen los métodos “registrarLlamadaNacional()” y “registrarLlamadaInternacional()” en la clase Empresa, teniendo que eliminar la sentencia “llamadas.add(llamada)”.

Antes Clase Empresa:

```
34
35     public Llamada registrarLlamadaNacional(Cliente origen, Cliente destino, int duracion) {
36         Llamada llamada = new LlamadaNacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
37         llamadas.add(llamada);
38         origen.registrarLlamada(llamada);
39         return llamada;
40     }
41
42     public Llamada registrarLlamadaInternacional(Cliente origen, Cliente destino, int duracion) {
43         Llamada llamada = new LlamadaInternacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
44         llamadas.add(llamada);
45         origen.registrarLlamada(llamada);
46         return llamada;
47     }
```

Despues Clase Empresa:

```
34
35      public Llamada registrarLlamadaNacional(Cliente origen, Cliente destino, int duracion) {
36          Llamada llamada = new LlamadaNacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
37          origen.registrarLlamada(llamada);
38          return llamada;
39      }
40
41      public Llamada registrarLlamadaInternacional(Cliente origen, Cliente destino, int duracion) {
42          Llamada llamada = new LlamadaInternacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
43          origen.registrarLlamada(llamada);
44          return llamada;
45      }
```

21) Mal olor: Feature Envy

En los siguientes métodos de la clase Empresa se produce envidia de atributos ya que Empresa instancia objetos de la clase Llamada y luego se lo manda por parámetro a un Cliente, cuando en realidad el Cliente debería encargarse de instanciar el objeto ya que Empresa no conoce a la clase Llamada.

Clase Empresa:

```
34
35      public Llamada registrarLlamadaNacional(Cliente origen, Cliente destino, int duracion) {
36          Llamada llamada = new LlamadaNacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
37          origen.registrarLlamada(llamada);
38          return llamada;
39      }
40
41      public Llamada registrarLlamadaInternacional(Cliente origen, Cliente destino, int duracion) {
42          Llamada llamada = new LlamadaInternacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
43          origen.registrarLlamada(llamada);
44          return llamada;
45      }
```

Refactoring: Extract Method

La solución es mover la instanciación de la llamada a la clase Cliente. Esto va a provocar que se tenga que modificar el método “registrarLlamada()” de la clase Cliente ya que, para poder diferenciar los dos tipos de Llamada es necesario dividir el método en “registrarLlamadaNacional()” y “registrarLlamadaInternacional()”, estos métodos en lugar de retornar “void” pasaran a retornar una instancia de la clase Llamada .

Despues Clase Empresa:

```
34
35      public Llamada registrarLlamadaNacional(Cliente origen, Cliente destino, int duracion) {
36          return origen.registrarLlamadaNacional(origen, destino, duracion);
37      }
38
39      public Llamada registrarLlamadaInternacional(Cliente origen, Cliente destino, int duracion) {
40          return origen.registrarLlamadaInternacional(origen, destino, duracion);
41      }
```

Antes Clase Cliente

```
27
28      public void registrarLlamada(Llamada llamada) {
29          llamadas.add(llamada);
30      }
```

Despues Clase Cliente:

```
27
28      public Llamada registrarLlamadaNacional(Cliente origen, Cliente destino, int duracion) {
29          Llamada llamada = new LlamadaNacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
30          llamadas.add(llamada);
31          return llamada;
32      }
33
34      public Llamada registrarLlamadaInternacional(Cliente origen, Cliente destino, int duracion) {
35          Llamada llamada = new LlamadaInternacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
36          llamadas.add(llamada);
37          return llamada;
38      }
```

22) Mal olor: Dead Code

En la clase Cliente podemos ver que ahora tenemos los métodos

“registrarLlamadaNacional()” y “registrarLlamadaInternacional()” los cuales reciben como parámetro “Cliente origen” el cual ya no es necesario porque el origen pasaría a ser el mismo Cliente que realizaría la instancia de la clase Llamada

Clase Cliente:

```
27
28      public Llamada registrarLlamadaNacional(Cliente origen, Cliente destino, int duracion) {
29          Llamada llamada = new LlamadaNacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
30          llamadas.add(llamada);
31          return llamada;
32      }
33
34      public Llamada registrarLlamadaInternacional(Cliente origen, Cliente destino, int duracion) {
35          Llamada llamada = new LlamadaInternacional(origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
36          llamadas.add(llamada);
37          return llamada;
38      }
```

Refactoring: Remove Parameter

En este refactoring resolvemos simplemente eliminando el parámetro que ya no esta en uso

Despues Clase Cliente:

```
27
28      public Llamada registrarLlamadaNacional(Cliente destino, int duracion) {
29          Llamada llamada = new LlamadaNacional(getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
30          llamadas.add(llamada);
31          return llamada;
32      }
33
34      public Llamada registrarLlamadaInternacional(Cliente destino, int duracion) {
35          Llamada llamada = new LlamadaInternacional(getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
36          llamadas.add(llamada);
37          return llamada;
38      }
```

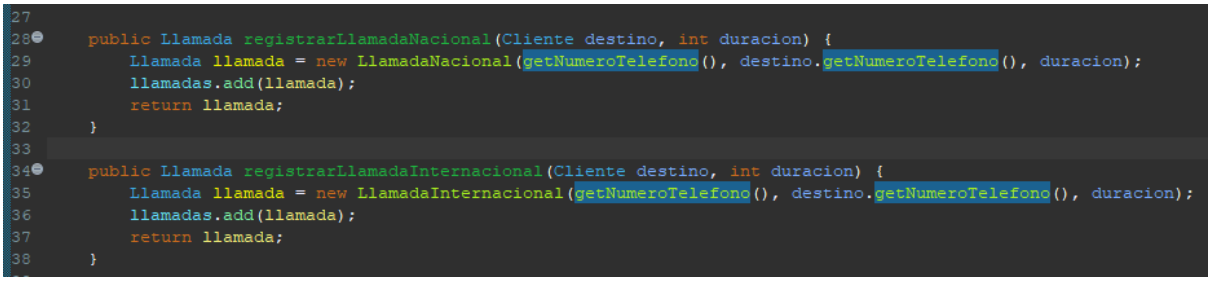
Despues Clase Empresa:

```
34
35      public Llamada registrarLlamadaNacional(Cliente origen, Cliente destino, int duracion) {
36          return origen.registrarLlamadaNacional(destino, duracion);
37      }
38
39      public Llamada registrarLlamadaInternacional(Cliente origen, Cliente destino, int duracion) {
40          return origen.registrarLlamadaInternacional(destino, duracion);
41      }
```

23) Mal olor: Duplicate Code

En este caso vemos como, ambos metodos “registrarLlamadaNacional()” y “registrarLlamadaInternacional()” de la clase Cliente agregan una llamada a la lista duplicando codigo.

Clase Cliente:

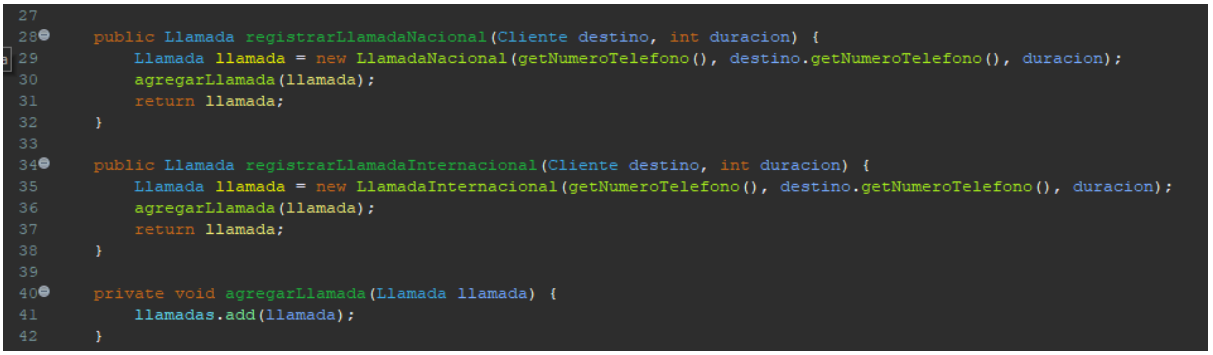


```
27
28      public Llamada registrarLlamadaNacional(Cliente destino, int duracion) {
29          Llamada llamada = new LlamadaNacional(getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
30          llamadas.add(llamada);
31          return llamada;
32      }
33
34      public Llamada registrarLlamadaInternacional(Cliente destino, int duracion) {
35          Llamada llamada = new LlamadaInternacional(getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
36          llamadas.add(llamada);
37          return llamada;
38      }
```

Refactoring: Extract Method

Procedemos a resolver este mal olor extrayendo en un método privado “agregarLlamada()”

Clase Cliente:

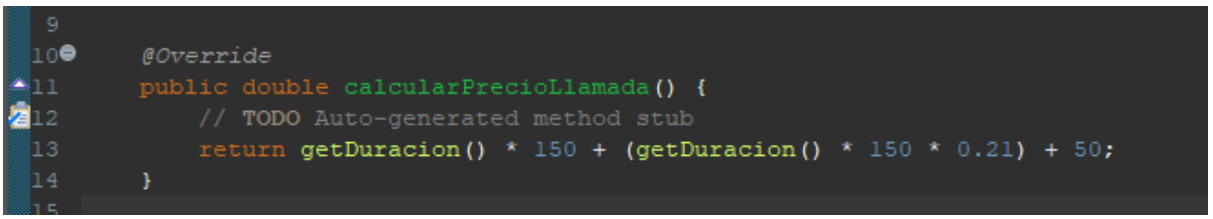


```
27
28      public Llamada registrarLlamadaNacional(Cliente destino, int duracion) {
29          Llamada llamada = new LlamadaNacional(getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
30          agregarLlamada(llamada);
31          return llamada;
32      }
33
34      public Llamada registrarLlamadaInternacional(Cliente destino, int duracion) {
35          Llamada llamada = new LlamadaInternacional(getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
36          agregarLlamada(llamada);
37          return llamada;
38      }
39
40      private void agregarLlamada(Llamada llamada) {
41          llamadas.add(llamada);
42      }
```

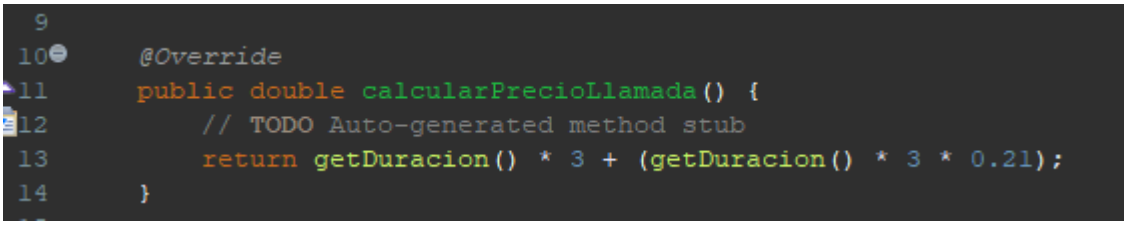
24) Mal olor: Duplicate Code

En la clase LlamadaInternacional y LlamadaNacional, tenemos el método BcalcularPrecioLlamada(), en ambos podemos ver como se repite una serie de pasos que podemos interpretar como calcularPrecioBase(), calcularIva() y calcularAdicional()

Clase LlamadaInternacional:



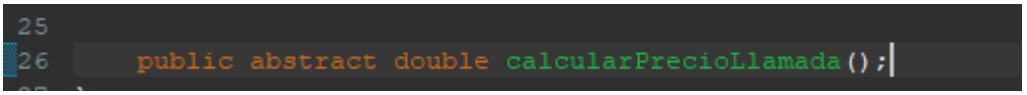
Clase LlamadaNacional:



Refactoring: Form Template Method

La solución a este mal olor es realizar uin Template Method, para eso, haremos que el método calcularPrecioLlamada() este implementado en la clase padre Llamada indicando los pasos this.calcularPrecioBase(), this.calcularIva() y this.calcularAdicional(), los ultimos serán métodos abstractos, que serán implementados en cada subclase LlamadaNacional y LlamadaInternacional

Antes Clase Llamada:



Despues Clase Llamada:

```
25
26      public double calcularPrecioLlamada() {
27          return this.calcularPrecioBase() +
28                  this.calcularIVA() +
29                  this.calcularAdicional();
30      }
31
32      protected abstract double calcularPrecioBase();
33
34      protected abstract double calcularIVA();
35
36      protected abstract double calcularAdicional();
37  }
```

Clase LlamadaInternacional:

```
 9
10      @Override
11      protected double calcularPrecioBase() {
12          // TODO Auto-generated method stub
13          return getDuracion() * 150;
14      }
15
16      @Override
17      protected double calcularIVA() {
18          // TODO Auto-generated method stub
19          return getDuracion() * 150 * 0.21;
20      }
21
22      @Override
23      protected double calcularAdicional() {
24          // TODO Auto-generated method stub
25          return 50;
26      }
27
```

Clase LlamadaNacional:

```
 9
10      @Override
11      protected double calcularPrecioBase() {
12          // TODO Auto-generated method stub
13          return getDuracion() * 3;
14      }
15
16      @Override
17      protected double calcularIVA() {
18          // TODO Auto-generated method stub
19          return getDuracion() * 3 * 0.21;
20      }
21
22      @Override
23      protected double calcularAdicional() {
24          // TODO Auto-generated method stub
25          return 0;
26      }
27
```

25) Mal olor: Nombre No Representativo

En la clase Empresa, el método agregarNumeroTelefono() recibe un parámetro “str” que hace referencia a un número de teléfono

Clase Empresa:

```
10
11      public boolean agregarNumeroTelefono(String str) {
12          return guia.agregarNumeroTelefono(str);
13      }
```

Refactoring: Rename Parameter

En este caso resolvemos cambiando el nombre del parémetro a “numeroTelefono”

Clase Empresa:

```
10
11      public boolean agregarNumeroTelefono(String numeroTelefono) {
12          return guia.agregarNumeroTelefono(numeroTelefono);
13      }
```

26) Mal olor: Nombre No Representativo

En la clase Empresa, el método registrarNumeroFisico() recibe un parámetro “data” que hace referencia a un dni e internamente tiene una variable “var” que hace referencia a un Cliente

Clase Empresa:

```
18
19      public Cliente registrarUsuarioFisico(String data, String nombre) {
20          ClienteFisico var = new ClienteFisico(nombre, this.obtenerNumeroLibre(), data);
21          agregarUsuario(var);
22          return var;
23      }
```

Refactoring: Rename Parameter

En este caso, resolvemos cambiando el nombre del parametro a “dni” y a la variable le llamamos “cliente”

Clase Empresa:

```
18
19      public Cliente registrarUsuarioFisico(String dni, String nombre) {
20          ClienteFisico cliente = new ClienteFisico(nombre, this.obtenerNumeroLibre(), dni);
21          agregarUsuario(cliente);
22          return cliente;
23      }
```

27) Mal olor: Nombre No Representativo

En la clase Empresa, el método registrarNumeroJuridico() recibe un parámetro “data” que hace referencia a un cuil e internamente tiene una variable “var” que hace referencia a un Cliente

Clase Empresa:

```
24
25      public Cliente registrarUsuarioJuridico(String data, String nombre) {
26          ClienteJuridico var = new ClienteJuridico(nombre, this.obtenerNumeroLibre(), data);
27          agregarUsuario(var);
28          return var;
29      }
```

Refactoring: Rename Variable

En este caso, resolvemos cambiando el nombre del parametro a “cuil” y a la variable le llamamos “cliente”

Clase Empresa:

```
24
25      public Cliente registrarUsuarioJuridico(String cuit, String nombre) {
26          ClienteJuridico cliente = new ClienteJuridico(nombre, this.obtenerNumeroLibre(), cuit);
27          agregarUsuario(cliente);
28          return cliente;
29      }
```

28) Mal olor: Nombre No Representativo

En la clase Cliente, ClienteJuridico y ClienteFisico, en el mètodo aplicarDescuento() tenemos un parametro “auxc” que hace referencia a un precio

Clase Cliente:

```
50
51      protected abstract double aplicarDescuento(double auxc);
52
```

Clase ClienteJuridico:

```
21
22      @Override
23      protected double aplicarDescuento(double auxc) {
24          // TODO Auto-generated method stub
25          return auxc - auxc*descuentoJur;
26      }
```

Clase ClienteFisico:

```
22      @Override
23      protected double aplicarDescuento(double auxc) {
24          // TODO Auto-generated method stub
25          return auxc - auxc*descuentoFis;
26      }
```

Refactoring: Rename Variable

En este caso resolvemos renombrando al parametro como “precio”

Clase Cliente:

```
50
51      protected abstract double aplicarDescuento(double precio);
52
```

Clase ClienteJuridico:

```
21
22      @Override
23      protected double aplicarDescuento(double precio) {
24          // TODO Auto-generated method stub
25          return precio - precio*descuentoJur;
26      }
```

Clase ClienteFisico:

```
22      @Override
23      protected double aplicarDescuento(double precio) {
24          // TODO Auto-generated method stub
25          return precio - precio*descuentoFis;
26      }
```

29) Mal olor: Nombre No Representativo

En la clase GestorNumerosDisponibles, tenemos el metodo cambiarTipoGenerador() donde se recibe el parametro “valor” que hace referencia a una instancia de TipoGenerador.

Clase GestorNumerosDisponibles:

```
24      public void cambiarTipoGenerador(TipoGenerador valor) {
25          this.tipoGenerador = valor;
26      }
```

Refactoring: Rename Parameter

En este caso resolvemos renombrando el parametro como “tipoGenerador”

Clase GestorNumerosDisponibles:

```
23
24      public void cambiarTipoGenerador(TipoGenerador tipoGenerador) {
25          this.tipoGenerador = tipoGenerador;
26      }
```

30) Mal olor: Nombre No Representativo

En la clase GestorNumerosDisponibles, tenemos el metodo agregarNumeroTelefono() donde se recibe el parametro “str” que hace referencia a un numero de telefono.

Clase GestorNumerosDisponibles:

```
27
28      public boolean agregarNumeroTelefono(String str) {
29          if (!getLineas().contains(str)) {
30              getLineas().add(str);
31              return true;
32          }
33          else {
34              return false;
35          }
36      }
```

Refactoring: Rename Parameter

En este caso resolvemos renombrando el parametro como “numeroTelefono”

Clase GestorNumerosDisponibles:

```
27
28      public boolean agregarNumeroTelefono(String numeroTelefono) {
29          if (!getLineas().contains(numeroTelefono)) {
30              getLineas().add(numeroTelefono);
31              return true;
32          }
33          else {
34              return false;
35          }
36      }
```